Theoretical and technical post-mortem

1-2 pages (500-1000 words)

Analyze your prototype via the course readings

Based on previously discussed lecture content, my prototype feels responsive and physical because every input produces consequences which can be either seen or heard instantaneously after the action is played. Pressing a key changes the animation blend, as well as the camera on occasion, smoothly so the player feels properly immersed instead of like a floating camera.You could think of it as a system of feedback loops with the controller telling the player what happened (landing thud/camera shake), what is possible (the tests gym has specific features), and finally what state the character is in (camera lowers and tightens when character is crouched). I wanted to avoid creating a rigid controller, so to do this I tightened the gravity and jump speed, and also added coyote time and jump buffering so inputs are forgiven. I also discovered the importance of clearing leftover physics velocity because of a hurdle I ran into about the player getting stuck in the crouch state even after I let go of the assigned key.

Core engine systems utilized

I used a variety of core engine systems in order to construct both the test gym environment and the character controller, starting with the new input system to make the character movable by both the keyboard and gamepad. I also downloaded Cinemachine using the package manager to create the third person effect. I also used ProBuilder and the readily available shapes to create the grey-box test gym environment.

Once it was understood, the New Input System was very helpful for assigning movements to keys for both keyboard and gamepad gameplay. It automatically handled the logistics of controlling the character with both keyboard/mouse and gamepad via one asset. Therefore, the various commands used on the controller (Move, Look, Sprint, Jump, Crouch) each have bindings for both devices mentioned in the criteria of this milestone. Most actions use the default Send Messages behavior, while crouch reads the action's state directly each frame.

The Cinemachine package was installed and used to set up state-driven freelook cameras to follow the player around and adapt the camera's height to match the state changes. The two distinct action states I chose for this milestone were crouching and jumping, both for which I used Cinemachine to visualize their differing orbit radius and field of view states. Both cameras track the character's head bone and use Basic Multi Channel Perlin noise for a subtle breathing sway. A Cinemachine Impulse Source triggers a camera shake on each jump landing, scaled by fall speed.

The animator controller uses two 2D blend trees driven by MoveX and MoveY, the first of which was for idle, walk, run and strafe movement. The second one handled creating smooth

crouch movements. I also included a single state for Jump, also facilitated by the same animator controller. The Rigidbody and physics layers on the character handled movement, ground detection (via layers) and headroom checks.

Most difficult hurdle when translating your design intention into C# or Unity components (+ resolution)

The most difficult hurdle I faced when putting my ideas into Unity was getting my character to only crouch while the C key was being pressed. For many tedious run throughs, the character would get stuck in crouch after one press because the console wasn't registering that the C key was released. I started by checking the Input Actions asset, which was not the issue. The 2D blend tree was functioning properly, so it had to be a problem with the character controller. The fix was to stop relying on the message callback and instead poll the Crouch action every frame, which cannot miss a release. I then added a headroom check, so the character only stands when nothing is above her. This struggle taught me how to separate the three places where a state can fail (input, code, animation) and test each one before changing anything.

Analyze the prototype through a lens framework

My prototype heavily reflects Steve Swink's model of game feel (2009), as most of the final decisions I made on the Unity build were thinking about how the response would play out. Swink's context metric is the most important since my prototype heavily focuses on the test gym I created for the character controller to explore. One of Swink's biggest arguments is that a character's feel is only relevant when tested in a space. My character's reaction times needed to feel immediate and present, so I added in things like coyote time and jump buffering to deal with mistimings that made the experience feel out of sync.

The audio-visual cues I had to add coincide with Swink's ideas about polishing. Syncing the landing thud and camera shake effect with the jump fall speed made it easier for the character to feel the impact of each jump and make it easier for the player to feel immersed.

Moment when Unity's architecture forced me to compromise on my original design

The part of this Unity build which created the most road blocks was undoubtedly setting up the crouch function. A moment that forced me to change a part of my original design was when I was trying to fit the character through a low pipe tunnel. I had initially assumed that the character's body would physically shrink, but the animation only moves the bones of the visible mesh, while the physics engine simulates a rigid capsule collider that stays at standing height regardless of how much space the character was taking up.

To fix this I had to abstract her body into a capsule and resize it in the code. My script ended up shortening the capsule by a ratio, shifting it so her feet stay on the ground, and running a capsule check before standing so she can't stand inside a tunnel. Unfortunately the collider changes instantly while the animation blends over time, so the physical body and the visible

body still briefly disagree. I accepted that because my priority was having a consistent rule about what fits.

Credits and References

Character
Erika Archer. Mixamo (Adobe), https://www.mixamo.com. Accessed 9 October 2026.

Animations (all from Mixamo, applied to Erika Archer)
Unarmed Idle, Walking, Running, Running Backward, Right Strafe, Right Strafe Walk, Left Strafe, Left Strafe Walk, Jump, Crouch Walk Right, Crouch Walk Left,
Crouch Walk Forward, Crouch Walk Back, Crouch Idle.

Audio
"Classic Footstep SFX" by Matthew Anett. Unity Asset Store, version 1.0
(released 3 July 2020).
https://assetstore.unity.com/packages/audio/sound-fx/classic-footstep-sfx-173668
Licensed under the Standard Unity Asset Store EULA. Accessed 9 October 2026.

Software and packages
Unity 6.3 LTS (6000.3.10f1); Cinemachine; Input System; ProBuilder.

Original work
Animator Controller and blend trees, PlayerMovement script, Cinemachine camera setup, and grey-box test gym were created by the author.

License note
Mixamo characters and animations are used under Adobe's Mixamo terms
(free for use in projects with an Adobe account).